For base system OTA rather than workloads, Mender, RAUC and SWUpdate all use A/B partitions with automatic rollback on failed boot.

Tier 4: Seeing a fleet you cannot reach

Bandwidth shapes everything. Shipping full Prometheus scrapes from every device is not viable over metered links, so the pattern is local aggregation, push to a collector, and sample rather than complete.

Signal	Frequency	Why
Heartbeat with model version	1 to 5 min	What is deployed is never quite what you think
Inference latency p50, p95	5 min, pre-aggregated	Hardware degradation shows here first
Input distribution summary	Hourly	Drift detection, the most valuable edge signal
Confidence distribution	Hourly	A falling mean is your early warning
Resource pressure	5 min	Thermal throttling is common and otherwise invisible
Full inference samples	0.1% or lower	For the retraining set

OpenTelemetry handles transport, and a collector on the device with batching and retry turns an offline period into a backlog rather than a hole.

Input drift is how edge AI dies. The model is trained on one plant in one season. Camera angles shift, lighting gets replaced, a supplier changes a component colour. None of this throws an error or moves a latency metric. A statistical summary of input features compared against the training distribution catches it weeks before a human does. Evidently, NannyML works well on aggregated summaries at tier 4.

Security

The edge threat model differs in one way that matters: the attacker may have physical access.

The practical minimum is secure boot with signed images, device identity rather than shared secrets (SPIFFE and SPIRE give each device short-lived credentials, where a shared key baked into four thousand images is one stolen device away from a bad week), signed model artefacts verified with Cosign before load, mutual TLS to the control plane, and encrypted local storage for buffered data.

None of it is exotic. All of it is far easier to build in than to retrofit across a deployed fleet.

Cost is bandwidth

You already bought the compute, so the recurring number is data movement. Model distribution is large and spiky but controllable through delta updates. Telemetry is moderate and constant, and controllable through aggregation. Raw data for retraining is the one that surprises people.

That last is the trap. Someone collects training data from production, nobody sets a sampling rate, and a thousand cameras start uploading frames. Put the rate in tier 3 config, adjustable without redeploy, with an alert on aggregate egress.

Delta updates are worth the engineering. A 400MB model to four thousand devices is 1.6TB. A binary diff against the cached previous version is frequently a tenth of that.

Build order

- Weeks 1-3: One device under GitOps control. Nothing else changes.
- Weeks 4-6: Separate model from image. Registry, version selector, cache.
- Weeks 7-9: Telemetry. Heartbeat, latency, drift summaries, a dashboard showing deployed model version per device.
- Weeks 10-12: Staged rollout machinery. Test the halt by deliberately shipping a bad model to the canary.

Teams that scale first and build the platform afterwards end up doing both at once, under pressure, with devices already in the field.

Where this leaves us

The manufacturing client is at eleven plants rather than forty-two, which is slower than planned and the right call. They spent a quarter building tiers 2 and 3 against a handful of devices, and the rollout since has been uneventful.

What convinced them was not architecture. It was the first staged rollout halting automatically at canary because confidence dropped on two of five devices, both at a plant with different overhead lighting. Across forty-two sites simultaneously, it would have been weeks before anyone connected the defect reports to the deployment.

That is the case for the platform layer, and it is the same case I made in September. The hard problem stopped being the model a while ago. It is running a thousand copies of it in places you cannot reach, without losing track of what they run and whether it still works.

Every component here is open source and most run on hardware you already own. Start with one device under GitOps control. That is a weekend, and everything else builds on it. END

By: Vishal Tanna

The author is the CEO of TechCrave, a technology services company that helps enterprises adopt open source, cloud native, and AI-driven solutions. He has over 15 years of experience in software development and technology leadership, and is particularly interested in scalable architectures and digital transformation.